

## 第7章 多态性与虚函数

在面向对象的程序设计中,另一个重要特征就是多态性。面向对象程序设计的四个重要特征是相互关联的。封装性是基础,继承性是关键,多态性是补充,抽象性则贯穿始终。本章将在介绍多态性概念的基础上,重点阐明运算符重载的实现方法和虚函数的使用。多态性存在于继承的环境下,它通过在不同类中定义同名成员函数,根据提供的公共基类去访问各种不同类的函数,从而实现一个接口和多种功能。通过本章的学习,可以更深刻地体会到面向对象程序设计比结构化程序设计具有更优越的性质和更强的生命力。

学习本章后,应了解多态性、运算符重载、静态联编和动态联编、虚函数、纯虚函数、抽象类等面向对象程序设计的基本概念;能够实现简单的运算符重载;掌握虚函数的定义和通过虚函数、根基类的指针和引用实现动态联编的原理以及动态联编的执行过程。

### 7.1 多态性概述

多态性是面向对象程序设计的一个重要特征。

所谓多态性,是指发出同样的消息被不同类型的对象接收时导致的不同行为。所谓发消息就是调用类中的成员函数。具体来说,多态是指类中具有相似功能的不同函数使用同一个名称。当调用这个相同名称的函数时,可根据需要完成不同的功能,而将所有的实现细节留给接收到消息的对象。利用多态性,程序员用向一个对象发送消息的方式来完成一系列动作,而不必涉及软件系统是如何实现这些动作的。这种方法更像解决实际问题的思维方式。

从概念方面讲,C++中的多态性可以分为四类,即参数多态、包含多态、重载多态和强制多态。前两种统称为通用多态,后两种统称为专用多态。

参数多态是指由函数模板和类模板的实例化实现的多态。模板中所定义的操作是相同的,但操作对象的类型可以不同。

包含多态是指由虚函数对各类中同名成员函数实现的多态。

重载多态是指由函数重载、运算符重载等实现的多态。

强制多态是指将一个变元的类型强制加以转换,以符合一个函数或操作的要求而实现的多态。例如,一个整型数和一个实型数在运算之前就必须进行类型强制转换。

从实现时机方面讲,多态性可以分为静态多态性和动态多态性两种。

静态多态性是指可在编译期间确定的多态性,通常称为静态联编(static binding)。重载多态和参数多态一般是静态多态的。

动态多态性是指在程序运行过程中才能确定的多态性,通常称为动态联编(dynamic binding)。包含多态和强制多态一般是动态联编的。

联编就是使计算机程序彼此关联的过程,也就是将一个函数标识符和一个存储地址联系在一起的过程。一般来说,静态联编有执行速度快的优点,而动态联编有灵活和高度抽象的优点。

关于参数多态、强制多态和函数重载,前面已经介绍过。本章将介绍运算符重载和虚函数实现的多态。

### 7.2 运算符重载

运算符重载是对类的对象进行的重载。运算符重载使程序员可以为自定义的新类型将已有的运算符赋予新的专门含义,从而使C++具有更好的扩充性。这是C++最具吸引力的特点之一。

一般来说,完成同样的操作,使用运算符要比调用函数更便于编写程序,也使程序更容

易阅读。

### 7.2.1 运算符重载概述

C++中的大多数运算符都可以重载。但也有一部分运算符不能重载。另外，在重载运算符时还要遵照一定的规则。

#### 1.不可重载的运算符

C++中有五个运算符不可重载。它们是圆点运算符“.”、成员指针选择运算符“.”、域分辨运算符“::”、条件运算符“?”和长度运算符“sizeof”。除以上五个运算符外,其余运算符均可重载。

#### 2.重载运算符的规则

重载运算符的规则如下:

①只能重载 C++中已有的可重载的运算符,不能建立新的运算符,如 C++中没有像其他程序语言中的乘幂运算符“\*\*”,但是,不可能通过重载建立这样的运算符;

②重载运算符时不能改变原运算符操作数的个数、原有运算符的优先级和结合性,也不能改变原运算符对于内部基本类型对象的含义;

③C++内部已对所有对象重载了两个运算符,即赋值运算符“=”和取地址运算符“&”(“=”的含义是逐个拷贝对象的数据成员,“&”的含义是返回对象在内存中的地址);

④如果重载了某个运算符(如“=”),并不意味着重载了相关的运算符(如“+=”、“-=”等)。

### 7.2.2 运算符重载的本质

C++将各种运算符都处理成一个函数调用。如表达式“5+7”将被解释成“operator+(5,7)”。其中 operator 是专用的重载运算符函数的名字。由于 C++已为各种基本数据类型定义了函数 operator+()。所以,用户只需定义自定义类型的重载运算符函数 operator+(),即可实现“+”运算符的重载。

在面向对象的程序设计中,类和对象是软件组成的基本成分。如果要为各类对象重载运算符,可以利用成员运算符函数和友元运算符函数实现运算符的重载。

可见,运算符重载的本质是一种特殊函数的重载。

编译程序将使用函数重载的原则,寻找与参数类型相匹配的运算符函数。所以,运算符重载的含义必须清楚,不得有二义性。

### 7.2.3 重载为类的成员函数

将运算符重载函数说明为类的成员函数的格式如下。

在类体内重载运算符函数原型说明的一般形式为:

返回类型 operator 运算符(形参表);

在类体外定义重载运算符函数的一般形式为:

返回类型 类名::operator 运算符(形参表){...}

其中, operator 是定义运算符重载函数的关键字。

由于运算符重载函数为类的成员函数,而成员函数被调用时都隐含有一个指向当前对象的 this 指针作实参,所以,一个一元运算符重载为类的成员函数时,参数个数为 0(即形参表必须为空),一个二元运算符重载为类的成员函数时,参数个数必须为 1。

例 7.1 利用成员函数实现两个复数的加、减二元运算。

```
#include <iostream.h>
class complex {
private:
    double real, imag;           //复数的实部和虚部系数
public:
    complex(double r=0, double i=0){ real=r;imag=i;}
```

```

        complex operator+(const complex &a);    //两个复数相加运算的成员函数
        complex operator-( const complex &a);    //两个复数相减运算的成员函数
        void list();                            //显示一个复数
};
void complex::list()                            //显示一个复数
{ cout<<"("<<real;                            //输出实部
  if(imag>=0)cout<<"+";
  cout<<imag<<"i)";                          //输出虚部
}
complex complex::operator+( const complex &a)//两个复数相加运算的成员函数
{ return complex(real+a.real,imag+a.imag); }
complex complex::operator-( const complex &a)//两个复数相减运算的成员函数
{ return complex(real-a.real,imag-a.imag); }
void main()    //测试复数类相加、相减运算符的程序
{  complex ob1(-4,3),ob2(2,-5.5);
   complex  add=ob1+ob2;// 复数类对象相加 ob1+ob2 , 相当于
ob1.operator+( ob2 )
   ob1.list();cout<<"+";ob2.list();cout<<"=";add.list(); cout<<endl;
   complex  sub=ob1-ob2; // 复数类对象相加 ob1-ob2 , 相当于
ob1.operator-( ob2 )
   ob1.list();cout<<"-";ob2.list();cout<<"=";sub.list();cout<<endl;
}

```

输出结果为:

$(-4+3i)+(2-5.5i)=(-2-2.5i)$

$(-4+3i)-(2-5.5i)=(-6+8.5i)$

程序说明如下:

①类 `complex` 中有四个成员函数。成员函数 `complex(double r=0,double i=0)` 为构造函数,完成数据的初始化,各参数的缺省值均为 0;成员函数 `void list()` 用于显示输出一个复数对象的值;另两个成员函数分别用于定义“+”运算符重载和“-”运算符重载。

②运算符重载函数实现时前边有两个 `complex`,如:

```
complex complex::operator+( const complex &a)
```

中第一个 `complex` 是该函数的返回类型,第二个 `complex` 为类名。

③运算符“+”和“-”是二元运算符(也称为双目运算符),在运算符重载函数中需要两个操作数,而在重载“+”和“-”的成员函数的形参表中仅有一个参数 `&a`,例如:

```
complex operator+( const complex &a);
```

```
complex operator-( const complex &a);
```

以运算符“+”为例,当 `main()` 执行 `add=ob1+ob2` 时,编译系统自动把 `ob1` 作为隐含参数(又称自引用参数)并通过 `this` 指针隐含传递,而 `ob2` 作为显式参数传给 `a`。因此,`real+a.real` 和 `imag+a.imag` 中的 `real` 和 `imag` 为隐含参数 `ob1` 的数据成员,`a.real` 和 `a.imag` 成为显式参数 `ob2` 的数据成员。

同样 `sub=ob1-ob2` 中的 `ob1` 也作为隐含参数参加运算。

作为练习,读者可以补充实现两个复数的乘、除运算符的重载。

## 7.2.4 重载为友元函数

将运算符重载函数说明为类的友元函数分为两步。

先在类体内声明重载运算符函数原型，格式如下：

**friend** 返回类型 **operator** 运算符(形参表);

再在类体外定义重载运算符的友元函数，格式如下：

返回类型 **operator** 运算符(形参表) {...}

其中，**operator** 是定义运算符重载函数的关键字。

由于友元函数被调用时没有隐含指向当前对象的 **this** 指针作实参，所以，一个一元运算符重载为类的友元函数时，参数个数为 1；一个二元运算符重载为类的成员函数时，参数个数为 2。

以下程序将“+”和“-”重载为友元运算符函数，以实现复数运算。运算结果与用成员函数形式实现相同。

例 7.2 利用友元函数实现两个复数的加、减运算。

```
#include <iostream.h>
class complex {
private:
    double real, imag;           //复数的实部和虚部系数
public:
    complex(double r=0, double i=0){ real=r;imag=i;}
    //两个复数相加运算的友元函数声明
    friend complex operator+(const complex &a,const complex &b);
    //两个复数相减运算的友元函数声明
    friend complex operator-( const complex &a,const complex &b);
    void list();                 //显示一个复数
};
void complex::list()            //显示一个复数
{ cout<<"("<<real;              //输出实部
  if(imag>=0)cout<<"+";
  cout<<imag<<"i)";            //输出虚部
}
//两个复数相加运算的友元函数定义
complex operator+( const complex &a,const complex &b)
{ return complex(a.real+b.real,a.imag+b.imag);}
//两个复数相减运算的友元函数定义
complex operator-( const complex &a,const complex &b)
{ return complex(a.real-b.real,a.imag-b.imag);}
void main()                     //测试复数类相加、相减运算符的程序
{ complex ob1(-4,3),ob2(2,-5.5);
  complex add=ob1+ob2; //复数类对象相加 ob1+ob2，相当于 operator+(ob1,
ob2 )
  ob1.list();cout<<"+";ob2.list();cout<<"=";add.list(); cout<<endl;
  complex sub=ob1-ob2; //复数类对象相加 ob1-ob2，相当于 operator-(ob1,
ob2 )
  ob1.list();cout<<"-";ob2.list();cout<<"=";sub.list();cout<<endl;
}
```

该例题将“+”和“-”重载为友元函数。由于友元函数不是类的成员函数，不能使用 **this** 指针

指向的隐含参数,因此友元函数 `operator+()`和 `operator-()`必须有两个参数。

在上面给出的例 7.1 和 7.2 中,实现运算符重载的函数都只包含一个 `return` 语句。有人在编写这样的函数时,常常定义一个临时对象。以实现运算符重载的友元函数为例,可以写成:

```
complex operator+( const complex &a,const complex &b)
{
    complex temp;
    temp.real= a.real+b.real;
    temp.imag= a.imag+b.imag;
    return temp;
}
```

这两种形式的函数除了格式不同之外,一般认为只包含一个 `return` 语句的函数会有更高的执行效率。

通过以上讨论可知:

①对于单目运算符,当以成员函数形式重载运算符时没有参数,以友元函数形式重载运算符时有一个参数;

②对于双目运算符,当以成员函数形式重载运算符时有一个参数,以友元函数形式重载运算符时有两个参数;

③运算符的重载不改变运算符的优先级和结合性,也不改变使用操作符的语法,只是给已有的操作符以新的功能,例如“+”运算符不但可以重载为复数相加,而且也可以重载对字符串相加。

另外,选择用成员函数还是用友元函数实现运算符重载,主要取决于个人习惯和经验。

对于单目运算符,常常选择用成员函数;对于运算符`=`、`()`、`[]`、`->`,只能选择用成员函数;对于复合赋值运算符`+=`、`-=`、`/=`、`*=`等常常选择用成员函数。

对于其他运算符,如“+”、“-”、“\*”、“/”等常常选择用友元函数,以适应更多的计算形式。例如,对于复数类对象相加来说,用友元函数实现时表达式 `4.5+ob1` 可以正确计算,而用成员函数实现时表达式 `4.5+ob1` 是错误的。因为 `4.5` 是一个基本类型的常数,不能自动转换为复数类对象。

### 7.2.5 几个常用运算符的重载

下面用例题说明几个常用运算符重载的方法。

#### 1. ++和--运算符的重载

`++`和`--`运算符均可作为前置运算符和后置运算符。为了区别前置与后置,使用无参的函数形式表示前置运算,用带有形式化参数 `int`(但不得写出参数名)的函数表示后置运算。

用成员函数形式实现时,前置运算符函数的原型说明为:

```
类名 operator++();
```

```
类名 operator--();
```

后置运算符函数的原型说明为:

```
类名 operator++(int);
```

```
类名 operator--(int);
```

这里的`(int)`是伪整型参数,无实际意义。

用友元函数实现时,前置运算符函数的说明为:

```
friend 类名 operator++(类名 &);
```

```
friend 类名 operator--(类名 &);
```

后置运算符函数的原型说明为:

```
friend 类名 operator++(类名 &, int);
```

```
friend 类名 operator--(类名 &, int);
```

这里的(int)是伪整型参数,无实际意义。

例 7.3 重载++和--运算符示例。用成员函数形式重载++,用友元函数重载--。

```
#include <iostream.h>
class sample {
public:
    sample(int i=0,int j=0){x=i;y=j;}
    void print(){cout<<"x="<<x<<"y="<<y<<endl;}
    sample & operator++();           //成员函数形式重载前置++
    sample & operator++(int);        //成员函数形式重载后置++
    friend sample & operator--(sample &); //友元函数形式重载前置--
    friend sample & operator--(sample &, int); //友元函数形式重载后置--
private:
    int x,y;
};
sample & sample::operator++()
{ ++x; ++y; return *this; } //返回当前对象
sample & sample::operator++(int)
{ ++x; ++y; return *this; } //返回当前对象
sample & operator--(sample &r) //友元函数形式重载前置--
{ --r.x; --r.y; return r; }
sample & operator--(sample &r, int) //友元函数形式重载后置--
{ --r.x; --r.y; return r; }
void main()
{ sample s1(2,7),s2(0,4);
  cout<<"s1:\n";
  ++(++s1);s1.print();
  (s1++)++;s1.print();
  cout<<"s2:\n";
  --(--s2);s2.print();
  (s2--)--;s2.print();
}
```

输出结果为:

```
s1:
x=4,y=9
x=6,y=11
s2:
x=-2,y=2
x=-4,y=0
```

首先,在用成员函数实现运算符++以及其他一元运算符重载时,常常返回当前对象(用\*this表示);在用友元函数实现运算符++以及其他一元运算符重载时,常常返回被操作的参数对象。

另外,注意本例中各重载运算符的函数都是返回 sample 类的引用,所以在计算表达式 ++(++s1)、(s1++)++、--(--s2)和(s2--)--的值时,能够连续两次对 s1 和 s2 的当前值加 1 或

减 1。也就是说，当重载运算符的函数返回一个类的引用时，才能够实现对被操作对象(即当前对象)连续多次执行这些运算。

当然，无论++还是--，不是必须对被操作对象的当前值加 1 或减 1，而应视实际需要。

下面这个例子说明如果重载运算符的函数不是返回一个类的引用时将产生的后果。

例 7.4 ++运算符的重载。其中后置++不是返回一个类的引用，而且对被操作对象的当前值不是加 1，而是加 12。

```
#include <iostream.h>
class sample {
public:
    sample(int i=0){x=i;}
    void print(){cout<<"x="<<x<<endl;}
    sample & operator++();           //重载前置++，返回的是一个类的引用
    sample operator++(int);         //重载后置++，返回的不是一个类的引用
private:
    int x;
};
sample & sample::operator++()
{ ++x; return *this; }             //返回当前对象
sample sample::operator++(int)
{ sample t= *this; x+=12; return *this; } //返回临时对象
void main (){
    sample c(3);
    cout<<"c 的初始 x 值: "; c.print();
    ++c; cout<<"对 c 的当前 x 值加 1: "; c.print();
    ++(++c); cout<<"对 c 的当前 x 值连续两次加 1: "; c.print();
    c++; cout<<"对 c 的当前 x 值加 12: "; c.print();
    (c++)++; cout<<"c 的最后 x 值: "; c.print(); //第 2 次对临时对象 x 值加 12
}
```

输出结果为:

```
c 的初始 x 值: x=3
对 c 的当前 x 值加 1: x=4
对 c 的当前 x 值连续两次加 1: x=6
对 c 的当前 x 值加 12: x=18
c 的最后 x 值: x=30
```

从程序的输出结果可以看出，由于重载后置++的函数返回的不是一个 sample 类的引用，所以，尽管返回语句形式上仍然是 return \*this，但是在执行连续多次的后置++运算时，仅有第一次是对 c 对象的 x 值加 12，后续都是对不同的临时对象 x 值加 12。这就是返回引用的函数与返回普通对象的函数之间的实质差别。

## 2. 数组下标运算符[ ]的重载

例 7.5 重载字符数组下标运算符。

C++中的数组名是一种常指针。在程序运行时,如果不对数组下标是否越界进行检查,有时是不安全的。

本例是实现一种带下标越界检查并重载了字符数组下标运算符的程序。

```
#include <iostream.h>
```

```

class charArray {
public:
    charArray(int n=10);
    ~charArray(){delete[] Buff;}
    int getLength(){return Length;}
    char & operator[](int i);
private:
    char *Buff;
    int Length;
};

charArray::charArray(int n) { Length=n>0 ? n:10; Buff=new char[n]; }
char & charArray::operator[](int i)
{ static char ch=0;
  if(i<Length && i>=0)return Buff[i];
  else { cout<<"\n 下标越界.";return ch; }
}

void main()
{ int k;
  charArray str(8);
  char *s="character string";
  for(k=0;k<8;k++) str[k]=s[k];    //str[k]使用重载后含义
  for(k=0;k<9;k++) cout<<str[k];  //str[k]使用重载后含义
  cout<<endl;
  cout<<"字符数组长度="<<str.getLength()<<endl;
}

```

输出结果为:

```

characte
下标越界.
字符数组长度=8

```

在本例主函数的赋值表达式 `str[k]=s[k]` 中含有两个 `[k]`。因为 `s` 是一个基本类型字符型的指针，所以，对 `s[k]` 是按照 C++ 下标运算符的内部含义理解的。而 `str` 是一个自定义类 `charArray` 的对象，所以，对 `str[k]` 就按照重载后的下标运算符含义理解，即调用下标运算符重载函数 `char & operator[]()`。

### 3. 运算符=、==的重载

例 7.6 对整数数组重载 =、==运算符示例。

```

#include <iostream.h>
#include <stdlib.h>

class intArray {                                //整数数组类定义
public:
    intArray (int n=10)                          //构造函数
    { size =n;
      Buff=new int[size];
      for(int i=0;i<size;i++) Buff[i]=0;        //全体数组元素初始化为 0
    }
}

```



```

intArray (const intArray &);           //拷贝构造函数
~intArray () {delete []Buff;}          //析构函数
const intArray & operator=(const intArray &);
int GetSize () { return size; }        //获取数组大小
int operator ==(const intArray &);     //重载==运算符
int & operator [] (int);               //重载[]运算符
private:
    int *Buff;
    int size;
};
int & intArray ::operator [] (int i)     //重载[]运算符函数定义
{ if(i>=0 && i<size)return Buff[i];    //下标正常, 返回数组元素 Buff[i]
  cout<<"index out of range\n";        //下标越界, 结束程序执行
  exit(1);
}
const intArray & intArray::operator=(const intArray &r) //重载=运算符函数定义
{ if(this != &r)                          //如果不是自赋值
  { delete [] Buff;                        //释放原数组的存储空间
    size=r.size;                          //取得新数组大小
    Buff= new int [size];                 //分配新的数组存储空间
    for(int i=0;i<size;i++) Buff[i]=r.Buff[i]; //传送对应数组元素值
  }
  return *this;                          //允许连续赋值 x=y=z
}
intArray::intArray (const intArray &a) //拷贝构造函数定义
{ size=a.size;                            //取得数组大小
  Buff=new int[size];                     //分配数组存储空间
  for(int i=0;i<size;i++) Buff[i]=a.Buff[i]; //传送对应数组元素值
}
int intArray::operator ==(const intArray &r) //重载==运算符函数定义
{ if(size !=r.size)return 0;              //如果数组大小不等则为假
  for(int i=0;i<size;i++)                 //如果数组大小相等
    if(Buff[i]!=r.Buff[i]) return 0;      //而对应数组元素值不等则为假
  return 1;                              //如果数组大小相等且全体对应数组元素值相等则为真
}
void main (){
    int cnt;
    intArray a1(6),a2;
    int n=a2.GetSize ();
    cout <<"a1 初值: ";
    for (cnt=0;cnt<6;cnt++) cout<<a1[cnt]<<" "; //输出 a1 初值
    cout <<"\n 输入 a2: ";
    for (cnt =0; cnt<n;cnt++) cin>>a2[cnt]; //输入 a2
    intArray a3(a2);                      //调用拷贝构造函数

```

```

cout << "用 a2 初始化的 a3: ";
for (cnt=0;cnt<n;cnt++) cout<<a3[cnt]<<" ";
a1=a2; //数组赋值
cout << "\n 把 a2 赋给 a1: ";
for (cnt=0;cnt<a1.GetSize();cnt++) cout<<a1[cnt]<<" ";
cout<<endl;
if(a1==a2) cout<<"a1 与 a2 相等\n";
else cout<<"a1 与 a2 不等\n";
}

```

执行示例为:

```

a1 初值:0 0 0 0 0 0
输入 a2:1 4 7 2 9 0 8 6 3 5
用 a2 初始化的 a3:1 4 7 2 9 0 8 6 3 5
把 a2 赋给 a1:1 4 7 2 9 0 8 6 3 5
a1 与 a2 相等

```

本例程序中包括几个运算符重载。首先是赋值运算符“=”。它是使用非常广泛的运算符，只能被重载为成员函数，并且是不能被继承的。在一般情况下，系统为每个类都生成一个缺省的赋值运算符。在没有特殊处理的情况下，如对内存的动态分配等，只使用这个缺省的赋值运算符就足够了。当运算对象不是基本类型且有特殊处理要求时，应当给出运算符重载函数，以实现对该类型的赋值运算。赋值运算符重载函数的一般形式为：

返回类型 & operator = (形参表);

在赋值运算符重载函数中，if(this != &r)语句中的比较条件是为防止 s=s 的自身赋值而设计的，语句“delete[]Buff;”是为了赋新值前删除原有空间，释放完空间后再分配新内存空间，这样就能在赋值运算过程中保证对动态内存管理的正确性。

特别要注意的是，赋值运算符是唯一不能被继承的运算符函数，也不能用友元函数重载。

下标运算符重载只能使用成员函数重载，其形式为

type1 & operator[] (type2)

其中 type1 为数组类型，type2 为下标类型。

#### 4. 流插入运算符<<和流提取运算符>>的重载

例 7.7 对数组重载流插入运算符和流提取运算符，这样可以像 Fortran 等语言那样输入和输出整个数组的全体元素。

```

#include<iostream.h>
#include<iomanip.h>
#include<stdlib.h>
class Array {
public:
    Array(int n=10){ nums=n>0?n:10; elems=new int[nums];}
    ~Array(){delete []elems;}
    friend ostream & operator<<(ostream &,Array &);
    friend istream & operator>>(istream &,Array &);
private:
    int * elems;int nums;
};
//流提取运算符必须用友元函数重载

```

```

istream & operator>>(istream &in,Array &a)
{ cout<<"输入"<<a.nums<<"个整数:\n";
  for(int i=0;i<a.nums;i++) in>>a.elems[i];
  return in; //可连续输入 cin>>a>>b
}
//流插入运算符必须用友元函数重载
ostream & operator<<(ostream &out,Array &a)
{ int c=0;
  for( int i=0;i<a.nums;i++)
  { out<<setw(6)<<a.elems[i];
    if(++c%5==0)out<<endl;
  }
  out<<endl;
  return out; //可连续输出 cout<<a<<b
}
void main()
{ Array m(16);
  cin>>m;
  cout<<"数组 m:\n"; cout<<m;
}

```

输出结果为:

输入 16 个整数:

12 34 42 8 1 0 9 299 95 56 60 87 33 66 19 74

数组 m:

|    |    |     |    |    |
|----|----|-----|----|----|
| 12 | 34 | 42  | 8  | 1  |
| 0  | 9  | 299 | 95 | 56 |
| 60 | 87 | 33  | 66 | 19 |
| 74 |    |     |    |    |

首先要注意的是重载<<或>>运算符的函数返回结果必须是 `ostream` 流或 `istream` 流的引用。这样才能连续进行插入或者提取的运算。其次要注意的是程序中使用了 `ostream` 和 `istream` 流的引用作重载<<和>>运算符函数的第一个形参。这是因为该形参在函数执行结束时要被返回。最后要注意的是必须用有元函数实现运算符<<或>>的重载。

### 7.3 虚函数

虚函数是一种成员函数，而且是非 `static` 的成员函数。一个函数被说明或定义为虚函数后，说明它在派生类中可能有多种不同的实现。虚函数只能通过指针或引用所表示的对象来调用。构造函数不允许为虚函数，但析构函数允许为虚函数，以便能自动释放由指针或引用所表示的对象。

#### 7.3.1 引入虚函数的原因

例 7.8 不使用虚函数的静态联编示例。

```

#include <iostream.h>
const double PI=3.14159;
class Point{ //定义基类
public:

```

```

    Point(double i,double j){x=i;y=j;}
    double Area(){return 0.0;}
private:
    double x,y;                //点坐标
};
class Line:public Point{
    double x1,y1;              //增加终点坐标成员
public:
    Line(double i1,double j1,double i2,double j2):Point(i1,j1){x1=i2;y1=j2;}
    double Area(){ return 0.0; }
};
class Circle:public Point{
    double radius;             //增加新成员“半径”
public:
    Circle(double r,double i,double j):Point(i,j),radius(r){}
    double Area(){ return PI*radius*radius;}
};
class Rectangle:public Point{
    double height, width;      //增加新成员高度、宽度
public:
    Rectangle(double h,double w,double i,double j):Point(i,j){height=h;width=w;}
    double Area(){ return height*width;}
};
void fun(Point *p){cout<<p->Area()<<endl;}
void main()
{   Line lin(1,3,4,5);        //定义线对象
    fun(&lin);
    Circle cir(10,3,6);       //定义圆对象
    fun(&cir);
    Rectangle rec(7,8,2,5);    //定义矩形对象
    fun(&rec);
}

```

本程序中的函数 `fun()` 有一个形参 `Point *p`，希望使用这样的函数计算不同几何对象的面积。但是，该程序的运行结果却为：

```

0
0
0

```

这是为什么呢？根据前一章的赋值兼容性规则，函数 `fun()` 的形参 `Point *p` 指向的是各派生类对象中的基类 `Point` 部分，`Point` 对象的面积定义为 0。根据本章开头关于多态性的描述得知，这属于静态联编。

为了得到需要的计算结果，在 C++ 中引入了虚函数。

### 7.3.2 虚函数与动态联编

要使一个成员函数成为虚函数，只需要在它的返回类型之前加上关键字 `virtual`。虚函数的一般格式如下：

**virtual** 类型名 函数名(形参表)

函数体

如果在类体外给出虚函数的实现,则仅在类体内虚函数原型说明处带有关键字 **virtual**。

一个函数被说明或定义为虚函数后,表明它在派生类中可能有多种不同的实现,即重新定义。但是,在派生类中重新定义时,其函数原型(包括返回类型、函数名、参数个数和类型以及各参数顺序)都必须与基类中的原型完全相同,否则,将处理为静态联编。

在 C++ 中,如果派生类中的函数没有用 **virtual** 说明,但与基类的虚函数有相同的函数名、参数个数和参数顺序,有相同的返回类型或返回满足赋值兼容规则的指针或引用,就自动被确定为虚函数。当然最好用 **virtual** 说明虚函数。

虚函数只能通过根基类指针或引用所表示的对象调用才能实现动态联编。如果通过对象调用则处理为静态联编。

一个类的构造函数不允许为虚函数,但析构函数允许为虚函数,以便能自动释放由指针或引用所表示的对象。

例 7.9 动态联编示例。

```
#include <iostream.h>
const double PI=3.14159;
class Point{                                //定义根基类
public:
    Point(double i,double j){x=i;y=j;}
    virtual double Area(){return 0.0;}      //定义虚函数
private:
    double x,y;                             //点坐标
};
class Line:public Point{
    double x1,y1;                           //增加终点坐标成员
public:
    Line(double i1,double j1,double i2,double j2):Point(i1,j1){x1=i2;y1=j2;}
    virtual double Area(){ return 0.0; }    //定义虚函数
};
class Circle:public Point{
    double radius;                          //增加新成员“半径”
public:
    Circle(double r,double i,double j):Point(i,j),radius(r){}
    virtual double Area(){ return PI*radius*radius;} //重新定义虚函数的实现
};
class Rectangle:public Point{
    double height, width;                   //增加新成员高度、宽度
public:
    Rectangle(double h,double w,double i,double j):Point(i,j){ height=h;width=w;}
    virtual double Area(){ return height*width;} //重新定义虚函数的实现
};
void fun(Point *p) { cout<<p->Area()<<endl; } //根基类指针作形参
void main()
{ Line lin(1,3,4,5);                       //定义线对象
```

```

    fun(&lin);
    Circle cir(10,3,6);    //定义圆对象
    fun(&cir);
    Rectangle rec(7,8,2,5); //定义矩形对象
    fun(&rec);
}

```

运行结果为:

```

0
314.159
56

```

程序说明如下。

①各派生类中的虚函数重新定义时可以不加关键字 **virtual** 而由系统自动判断,但建议各虚函数均加 **virtual** 关键字,以提高程序的可读性。

②在 **main()** 中定义指向根基类的指针 **p** 可以指向派生类,在执行过程中,可根据指针 **p** 的当前指向而调用相应类的虚函数,从而实现动态的多态性。注意派生类的指针不能指向基类。

③函数 **fun()** 的形参也可以是根基类的引用而改成如下形式:

```
void fun(Point &p) { cout<<p.Area()<<endl; } //根基类的引用作形参
```

主函数也相应改成如下形式:

```

void main()
{ Line lin(1,3,4,5); //定义线对象
  fun(lin);
  Circle cir(10,3,6); //定义圆对象
  fun(cir);
  Rectangle rec(7,8,2,5); //定义矩形对象
  fun(rec);
}

```

### 7.3.3 虚析构造函数

在 C++ 中,不允许说明虚构造函数,但是可以说明虚析构造函数。其形式为:

```
virtual ~类名() 虚析构造函数体
```

如果一个类的析构函数为虚函数,则由它派生而来的所有派生类的析构函数均为虚析构函数。定义了虚析构函数后,使用根基类指针可实现动态联编,根据当前指针的指向调用适当的析构函数释放对象。一般用 **delete** 运算符删除动态分配的对象。

例 7.10 虚析构造函数使用示例。

```

#include <iostream.h>
class A{
public:
    A( int m=0) { x=m; }
    virtual ~A(){ cout<<"类 A 虚析构造函数\n";}
private:
    int x;
};
class B:public A{
public:
    B(int n,int m):A(m){ size=n; buf=new char[n]; }
}

```

```

~B(){ delete[]buf;cout<<"类 B 虚析构函数\n";}
private:
    char *buf;
    int size;
};
void fun(A*a){ delete a;}
void main()
{   A *a=new B(10,7);
    fun(a);
}

```

输出结果为:

类 B 虚析构函数

类 A 虚析构函数

尽管本例主函数中定义的是类 A 指针,但由于它指向的是动态分配创建的类 B 对象,因此仍按释放类 B 对象的方法先释放类 B 自身的成员,然后释放派生类 B 中所包含的基类 A 的成员。

### 7.3.4 纯虚函数与抽象类

在根基类中定义虚函数时必须给出一个函数体,但这个函数体并没有任何实际意义。这种情况下可以将该函数定义为纯虚函数。纯虚函数的形式为:

```
virtual 类型名 函数名(形参表)=0;
```

带有纯虚函数的类称为抽象类。

C++中对抽象类规定如下:

①抽象类只能作为其他类的基类,即根基类,不能建立抽象类的对象,但是可以说明抽象类的指针或引用,以便使用这些指针和引用实现动态多态性;

②抽象类不能作为函数返回类型、参数类型,也不能作为转换结果的类型;

③抽象类主要作为继承结构中的根基类,继承抽象类纯虚函数的派生类仍然是一个抽象类,只有在派生类中给出基类纯虚函数的实现时,该派生类才成为可以建立对象的具体类。

例 7.11 纯虚函数和抽象类的使用。

```

#include <iostream.h>
const double PI=3.1415926;
class Shape{                                //抽象类定义
public:
    virtual void PrintShapeName()=0;
    virtual double Area()=0;
    virtual double Volume()=0;
};
class Point:public Shape{                   //定义点类
public:
    Point(double a=0,double b=0){ x=a;y=b;}
    double GetX(){ return x;}
    double GetY(){ return y;}
    virtual double Area(){return 0.0;}
    virtual double Volume(){return 0.0;}
    virtual void PrintShapeName(){cout<<"点:";}
}

```

```

private:
    double x,y;
};
class Circle:public Point{           //定义圆类
public:
    Circle(double r=0.0,double a=0.0,double b=0.0):Point(a,b)
    { radius=r>0?r:0;}              //初始化为安全的值
    double Getradius(){ return radius; }
    double Area(){ return PI*radius*radius; }
    double Volume(){ return 0.0;}
    virtual void PrintShapeName(){ cout<<"圆:"; }
private:
    double radius;
};
class Sphere:public Circle{         //定义球体类
public:
    Sphere(double r=0.0,double a=0.0,double b=0.0):Circle(r,a,b){ }
    //球体没有特殊的数据成员
    virtual void PrintShapeName(){ cout<<"球体:"; }
    double Area(){ return Circle::Area()*4; }
    double Volume(){ return Area()*Getradius()/3.0; }
};
void print(shape *p1,Point &p2)
{ p1->PrintShapeName();
  cout<<"x="<<p2.GetX()<<",y="<<p2.GetY()
    <<" ,面积="<<p1->Area()<<" ,体积="<<p1->Volume()<<endl;
}
void main()
{ Point point(7,11); print(&point, point);
  Circle circle(10,8,32); print(&circle,circle);
  Sphere sphere(10,20,8); print(&sphere,sphere);
}

```

输出结果为:

点:x=7,y=11,面积=0,体积=0

圆:x=8,y=32,面积=314.159,体积=0

球体:x=20,y=8,面积=1 256.64,体积=4 188.79

## 7.4 例题

例 7.12 有一个人员信息管理类 `per`,将它作为基类,派生出学生类 `student`、教师类 `teacher` 和工人类 `worker`,定义一个虚函数用以显示各类信息。

```

#include <iostream.h>
class per{
protected:

```



```

        char *name;
        int age;
    public:
        virtual void message(){ cout<<"person message\n"; }
};
class teacher:public per{
    public:
        void message(){ cout<<"teacher message\n"; }
};
class student:public per{
    public:
        void message(){ cout<<"student message\n"; }
};
class worker:public per{
    public:
        void message(){ cout<<"worker message\n"; }
};
void main()
{
    per obj,*ptr;
    teacher ob1;
    student ob2;
    worker ob3;
    ptr=&obj;
    ptr->message();           //调用基类成员函数
    ptr=&ob1;
    ptr->message();           //调用 teacher 类成员函数
    ptr=&ob2;
    ptr->message();           //调用 student 类成员函数
    ptr=&ob3;
    ptr->message();           //调用 worker 类成员函数
}

```

运行结果为:

```

person message
teacher message
student message
worker message

```

例 7.13 编写程序对字符串类重载"+"运算符和"="运算符实现两个字符串的连接。

```

#include <iostream.h>
#include <string.h>
class cstring {
    char *ps;
    int n;
    public:
        cstring(){ n=0; ps=0; }

```

```

cstring(char *cp) { n=strlen(cp)+1; ps=new char[n]; strcpy(ps,cp); }
cstring & operator=(char *p)
{ delete ps; n=strlen(p)+1; ps=new char[n]; strcpy(ps,p); return *this; }
cstring & operator=(cstring &s)
{ delete ps; n=s.n; ps=new char[n]; strcpy(ps,s.ps); return *this; }
cstring & operator+(char *p)
{ char *p1;int n1=n; p1=new char[n1]; strcpy(p1,ps);
  n+=strlen(p)+1; delete ps; ps=new char[n]; strcpy(ps,p1);
  strcat(ps,p); return *this;
};
cstring & operator+(cstring &s)
{ char *p1;int n1=n; p1=new char[n1]; strcpy(p1,ps);
  n+=s.n; delete ps; ps=new char[n]; strcpy(ps,p1);
  strcat(ps,s.ps); return *this;
}
void disp(){ cout<<ps<<endl; }
};
void main()
{ cstring str1,str2,str3(" string class "),str4;
  char *pc=" test"; cout<<"pc="<<pc<<endl;
  str1=pc; cout<<"str1="; str1.disp();
  str2=str3; cout<<"str2="; str2.disp();
  cout<<"str3="; str3.disp();
  str4=str3+pc; cout<<"str3+pc="; str4.disp();
  str4=str1+str2; cout<<"str1+str2="; str4.disp();
}

```

运行结果为:

```

pc= test
str1= test
str2= string class
str3= string class
str3+pc= string class test
str1+str2= test string class

```

例 7.14 简单的本科生与研究生的成绩管理程序。

```

#include<iostream.h>
class Student{
    int exams; int * marks;char * name;
public:
    Student(int e,int * m,char * n){exams=e;marks=m;name=n;}
    int totalmark();
    virtual void format();
    void print();
};
int Student::totalmark()

```

```

{   int   sum=0 ;
    for(int i=0;i<exams; i++) sum+=marks[i];
    return  sum ;
}
void Student::format(){cout<<name<<" total="<<totalmark(); }
void Student::print()
{   cout<<"student " ;
    format();
    cout <<endl;
}
class gradStudent:public Student {
    char   * supervisor;
public:
    gradStudent(int e,int   * m,char *n,char * superv)
        :Student(e,m,n){supervisor=superv;}
    virtual void format();
};
void gradStudent::format()
{   Student::format();
    cout<<"\n\t(graduate,supervised   by " << supervisor << ")\n" ;
}
int s[]={90,70,80,60};
int g[]={60,88,72,92,80};
void main()
{   Student   st(4,s,"LiNa");st.print();
    gradStudent   gt(5,g,"GaoMing","LiYu");gt.print();
}

```

输出结果为:

```

student LiNa total=300
student GaoMing total=392
      (graduate,supervised   by LiYu)

```

## 练习 7

1. 回答下列问题:

- (1) 什么叫多态性, 多态性在 C++ 中分为哪几类?
- (2) 什么是运算符重载, 哪些运算符不能重载?
- (3) 在 C++ 中是如何实现运算符重载的, 用成员函数和友元函数重载运算符有何不同?
- (4) 什么虚函数, 虚函数与哪些 C++ 的成分结合可以实现动态联编?

2. 选择题。

- (1) 在下列运算符重载的描述中, \_\_\_\_\_ 是正确的。  
 A) 可改变操作数个数    B) 可改变优先级    C) 可改变结合性    D) 不可改变语法结构
- (2) 在下列纯虚函数和抽象类的描述中, \_\_\_\_\_ 是错误的。  
 A) 纯虚函数没有具体实现    B) 抽象类是具有纯虚函数的类  
 C) 抽象类的派生类一定不再是抽象类    D) 抽象类不能建立对象
- (3) 在下列动态联编的描述中, \_\_\_\_\_ 是错误的。  
 A) 动态联编是在编译时确定操作函数的  
 B) 动态联编是以虚函数为基础的  
 C) 动态联编必须通过指针或引用来调用虚函数  
 D) 动态联编是在运行时确定操作函数的
- (4) \_\_\_\_\_ 是一个在基类中说明的虚函数, 它在该基类中没有具体的实现, 但要求派生类必须有自己的版本。  
 A) 虚构造函数    B) 虚析构造函数    C) 纯虚函数    D) 静态成员函数
- (5) 若在一个类中至少有一个纯虚函数, 则称该类为 \_\_\_\_\_。  
 A) 虚基类    B) 多重派生类    C) 抽象类    D) 友元类

3. 阅读下列的程序并写出输出结果:

```
(1)  #include <iostream.h>
      class Base {
      public:
          virtual void fun(int x){ cout<<"Base class x="<<x<<endl;}
      };
      class subclass:public Base {
          virtual void fun(int x){ cout<<"subclass x="<<x<<endl; }
      };
      void Test (Base *p) { int a=2;  p->fun(a); }
      void main()
      {  Base obj1;
         subclass obj2;
         Test (&obj1);  Test (&obj2);
      }
```

```
(2)  #include <iostream.h>
      #include <math.h>
      class base          //抽象类 base
      {  protected:int a,n;
      public:
          virtual void setab(int i, int j=0){ a=i;n=j; }
```

```

    virtual void disp()=0; //纯虚函数
};
class pa1:public base    //派生类 pa1
{ public: void disp(){ cout<<a*a<<endl; } };
class pa2:public base    //派生类 pa2
{ public: void disp(){ cout<<a*a*a<<endl; } };
class pan:public base    //派生类 pan
{ public: void disp(){ cout<<pow(double(a),double(n))<<endl; } };
void main()
{   base *pa;
    pa1 ob1; pa2 ob2; pan ob3;
    pa=&ob1; pa->setab(10); pa->disp();
    pa=&ob2; pa->setab(5); pa->disp();
    pa=&ob3; pa->setab(5,4); pa->disp();
}

```

4.程序填空题。以下程序对点类用成员函数进行运算符“+”重载，用友元函数进行运算符“-”重载，以实现两个点坐标的相加、相减运算。

```

#include <iostream.h>
class point {
private: int x,y;
public:
    point(int i=0,int j=0){ x=i;y=j; }
    void display(){cout<<"("<<x<<","<<y<<")"<<endl;}
    point operator+( _____ ){ return point(x+p.x, y+p.y); }
    _____ point operator-( point &, point &);
};
point operator-( point &q, point &p){ return point(q.x-p.x, q.y-p.y); }
void main()
{   point p1(4,5),p2(3,9),p3;
    p3=p1+p2;    p3.display();
    p3=p1-p2;    p3.display();
}

```

5.用虚函数和抽象类的概念设计求正方体、圆柱体和球体体积的程序。

6.设计一个字符串类重载“+”运算符,使其实现两个字符串的连接并赋值。试分别用成员函数和友元函数实现。